



UNIVERSIDADE ESTADUAL DE MARINGÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

ENZO VIGNOTTI SABINO – RA 133791
NUNO MIGUEL MENDONÇA ABILIO – RA 132830

RELATÓRIO TÉCNICO DE TRABALHO PRÁTICO DA DISCIPLINA DE
MODELAGEM E OTIMIZAÇÃO ALGORÍTMICA

APLICAÇÃO DE META-HEURÍSTICAS DE ALGORITMOS
GENÉTICOS E BUSCA LOCAL PARA SOLUCIONAR PROBLEMA
DE COBERTURA DE CONJUNTOS

Universidade Estadual de Maringá – Departamento de Informática
Av. Colombo, 5790 – Bloco C56, Zona 7 – CEP: 87020-900
Fone: (44) 3011-4324/3011-4219
Maringá – PR
2026

ENZO VIGNOTTI SABINO – RA 133791
NUNO MIGUEL MENDONÇA ABILIO – RA 132830

**APLICAÇÃO DE META-HEURÍSTICAS DE ALGORITMOS
GENÉTICOS E BUSCA LOCAL PARA SOLUCIONAR PROBLEMA
DE COBERTURA DE CONJUNTOS**

*Relatório Técnico de Trabalho Prático apresentado ao
Departamento de Informática da Universidade Estadual
de Maringá como requisito parcial à aprovação na
disciplina de Modelagem e Otimização Algorítmica do
Curso de Bacharelado em Ciência da Computação.*

Prof. Dr. Ademir Aparecido Constantino.

Universidade Estadual de Maringá – Departamento de Informática
Av. Colombo, 5790 – Bloco C56, Zona 7 – CEP: 87020-900
Fone: (44) 3011-4324/3011-4219

Maringá – PR

2026

Sumário

	Páginas
1 Introdução	4
2 Descrição do Problema (SCP)	4
2.1 Definição formal	4
2.2 Representação matemática	5
2.3 Linhas, colunas, custos e cobertura	5
2.4 Relevância prática	5
3 Descrição do Algoritmo Proposto	5
3.1 Algoritmo de construção de solução	6
3.2 Representação dos dados de entrada	7
3.3 Representação interna da solução	8
3.4 Estruturas auxiliares	8
3.5 Operadores genéticos	9
3.6 Procedimentos de busca local	11
3.7 Estrutura de dados da população e substituição	11
3.8 Elementos de originalidade	12
4 Experimentos Computacionais	12
4.1 Ambiente de execução	12
4.2 Instâncias utilizadas	12
4.3 Parâmetros do AG e da busca local	13
4.4 Processo de calibração dos parâmetros	13
5 Resultados	16
5.1 Melhores soluções, custos, tempos e GAP	16
5.2 Boxplot do GAP por instância	17
5.3 Análise de convergência	18
6 Conclusões	20

1 Introdução

Este relatório descreve o projeto, a implementação e a avaliação experimental de um algoritmo para o **Problema de Cobertura de Conjuntos** (*Set Covering Problem*, SCP), desenvolvido no âmbito da 3ª Avaliação de 2026 da disciplina Modelagem e Otimização Algorítmica. O SCP é um problema clássico de otimização combinatória, com aplicações em áreas como escalonamento de tripulações, localização de instalações, cobertura de redes e problemas de decisão em logística.

A meta-heurística adotada para resolver o problema foi um **Algoritmo Genético (AG)** combinado com um procedimento de **Busca Local**. A escolha por essa combinação se justifica pela complementaridade entre as duas técnicas: enquanto o AG realiza uma exploração global do espaço de soluções por meio de operadores de recombinação (*crossover*) e mutação, aplicados sobre uma população de soluções candidatas, a busca local atua de forma complementar, refinando individualmente as soluções geradas em cada geração e acelerando a convergência para regiões de melhor qualidade do espaço de busca. Essa hibridização, comumente referida na literatura como Algoritmo Memético, tende a produzir soluções de melhor qualidade do que o uso isolado de qualquer uma das duas técnicas.

Nos experimentos computacionais buscou-se avaliar: (i) a qualidade das soluções obtidas em relação a valores de referência da literatura, medida pelo indicador GAP; (ii) o comportamento do algoritmo ao longo das gerações, por meio de uma análise de convergência; e (iii) o desempenho computacional (tempo de execução) para instâncias de diferentes portes.

A base do algoritmo foi retirada de (CONSTANTINO et al., 2003). Do ponto de vista da originalidade da implementação, destaca-se: um método aleatório de construção das soluções iniciais, uma busca local baseada em uma vizinhança de troca fundamentada na relação de dominância de cobertura entre colunas e um critério de parada híbrido que considera a estagnação da solução além do número máximo de iterações permitido. Esses elementos são detalhados na Seção 3.

2 Descrição do Problema (SCP)

2.1 Definição formal

O Problema de Cobertura de Conjuntos pode ser definido a partir de um conjunto universo de elementos (linhas) $I = \{1, \dots, m\}$ e uma coleção de subconjuntos (colunas) $J = \{1, \dots, n\}$, em que cada coluna $j \in J$ possui um custo $c_j > 0$ e cobre um subconjunto de elementos de I . O objetivo é selecionar um subconjunto $S \subseteq J$ de colunas, de custo total mínimo, tal que todo elemento de I seja coberto por pelo menos uma coluna de S .

2.2 Representação matemática

Seja $a_{ij} \in \{0, 1\}$ um coeficiente que indica se a linha i é coberta pela coluna j ($a_{ij} = 1$) ou não ($a_{ij} = 0$), e seja $x_j \in \{0, 1\}$ uma variável de decisão que indica se a coluna j é selecionada para compor a solução. O SCP pode ser formulado como o seguinte problema de Programação Linear Inteira:

$$\text{minimizar } \sum_{j=1}^n c_j x_j \quad (1)$$

$$\text{sujeito a } \sum_{j=1}^n a_{ij} x_j \geq 1, \quad \forall i \in I \quad (2)$$

$$x_j \in \{0, 1\}, \quad \forall j \in J \quad (3)$$

A Equação (1) representa a minimização do custo total das colunas selecionadas. A restrição (2) garante que cada linha i seja coberta por pelo menos uma coluna selecionada (relação de cobertura, não de particionamento, diferente do *Set Partitioning Problem*, redundâncias são permitidas, porém indesejáveis do ponto de vista do custo). A Equação (3) define o domínio binário das variáveis de decisão.

2.3 Linhas, colunas, custos e cobertura

No contexto do problema: cada **linha** representa um elemento que precisa ser coberto; cada **coluna** representa uma alternativa disponível, associada a um **custo** de utilização; e a **cobertura** é a relação binária entre colunas e linhas que define quais linhas cada coluna consegue atender. Uma solução viável é qualquer subconjunto de colunas cuja união das linhas cobertas seja igual ao conjunto total de linhas.

2.4 Relevância prática

O SCP modela uma ampla gama de problemas reais: escalonamento de tripulações aéreas e ferroviárias, localização de instalações de emergência, problemas de cobertura em redes de telecomunicações e sensores, entre outros. A relevância prática do problema, aliada à sua dificuldade computacional, torna-o um caso de uso emblemático para o estudo e aplicação de meta-heurísticas.

3 Descrição do Algoritmo Proposto

Esta seção descreve o algoritmo implementado no código-fonte (arquivos: `teste.py`, `cromossomo.py`, `source.py` e `utils.py`).

3.1 Algoritmo de construção de solução

```
1 populacao <- gerarPopulacaoInicial()
2
3 numIteracoes = 0
4 numEstagnadas = 0
5 while (numIteracoes <= iteracoesMaximas) do:
6     numEstagnadas += 1
7
8     reprodutores <- populacao.selecionaReprodutores()
9     filho <- cruzamento(reprodutores)
10    filho <- mutar(filho)
11    filho <- buscaLocal(filho)
12
13    menosApto <- populacao.getMenosApto()
14    se filho.custo < menosApto.custo:
15        populacao.remove(menosApto)
16        populacao.adiciona(filho)
17        numIteracoes += 1
18        numEstagnadas = 0
19
20    se numEstagnadas >= (iteracoesMaximas/4):
21        parar
```

Listing 1. Implementação da filtragem High-Boost e máscara de nitidez

A construção de cada indivíduo da população inicial é realizada pelo método `_gerarIndividuo()` da classe `Teste`, por meio de um procedimento randomizado, orientado por linha:

```
1 def _gerarIndividuo(self) -> Cromossomo:
2     genesEscolhidos = set()
3     linhasDescobertas = set([i for i in range(1, self.numLinhas+1)]
4         )
5     coberturaPorLinha = [0] * (self.numLinhas + 1)
6
7     while not (len(linhasDescobertas) == 0):
8         linha = random.choice(list(linhasDescobertas))
9         colunasQueCobrem = self._colunasPorLinha[linha]
10        melhorColuna = self._melhorColuna2(linha)
11        linhasDescobertas.remove(linha)
12
13        if melhorColuna != 0:
14            linhasCobertas = self._linhasPorColuna[melhorColuna]
```

```

14         genesEscolhidos = genesEscolhidos | {melhorColuna}
15         ...
16         linhasDescobertas = linhasDescobertas - linhasCobertas
17         ...
18         individuo = Cromossomo(genesEscolhidos)
19         return individuo

```

Listing 2. Implementação da filtragem High-Boost e máscara de nitidez

O procedimento sorteia repetidamente uma linha ainda não coberta e escolhe, dentre as colunas que cobrem essa linha, aquela apontada pela função `_melhorColuna2()` como a “melhor” candidata, adicionando-a à solução e atualizando o conjunto de linhas descobertas, até que todas as linhas estejam cobertas.

A escolha da “melhor coluna” para a linha sorteada é feita pelo método `_melhorColuna2()`:

```

1 def _melhorColuna2(self, linha=None):
2     coluna = 0
3
4     if linha is not None:
5         colunasQueCobrem = self.getColunasDaLinha(linha)
6         coluna = random.choice(list(colunasQueCobrem))
7
8         if len(colunasQueCobrem) == 0:
9             print(f'\n 0 Colunas Cobrem a Linha {linha}')
10    return coluna

```

Listing 3. Implementação da filtragem High-Boost e máscara de nitidez

Pelo método descrito acima, dada uma linha descoberta passada como parâmetro, são recuperadas todas as colunas que cobrem aquela linha e, dentre elas, uma é escolhida aleatoriamente como a “melhor” e retornada.

3.2 Representação dos dados de entrada

Os dados de entrada são armazenados como uma **lista de registros** (`self.dados : List[List[float]]`), na qual cada elemento é uma lista contendo o identificador da coluna, seu custo e a relação das linhas por ela cobertas. Essa lista é lida pela função `ExtraiDados()` do módulo `utils.py`, que interpreta o cabeçalho do arquivo (linhas LINHAS e COLUNAS) e, em seguida, faz o *parsing* linha a linha dos dados de cada coluna por meio da função auxiliar `ExtraiDadosLinha()`.

A partir dessa lista, três estruturas de acesso rápido baseadas em dicionários e conjuntos (`dict/set`) são construídas uma única vez, no carregamento da instância, pelos métodos `gerarPesoColunas()`, `gerarLinhasPorColuna()` e `gerarColunasPorLinha()`:

- `pesoColunas : Dict[int, float]`, mapeia cada coluna ao seu custo c_j ;
- `_linhasPorColuna : Dict[int, Set[int]]`, mapeia cada coluna ao conjunto de linhas que ela cobre;
- `_colunasPorLinha : Dict[int, Set[int]]`, mapeia cada linha ao conjunto de colunas que a cobrem (construída a partir da estrutura anterior, por inversão).

Ou seja, a representação de entrada adotada é uma **representação esparsa baseada em listas de adjacência** (dicionários de conjuntos), e não uma matriz binária densa $m \times n$. Essa escolha é apropriada para instâncias do SCP, tipicamente esparsas, pois evita o armazenamento e a varredura de um grande número de zeros e permite operações de interseção e diferença de conjuntos de forma eficiente, fundamentais tanto para a construção dos indivíduos quanto para a busca local, como discutido na Seção 3.4.

3.3 Representação interna da solução

Cada indivíduo da população é representado pela classe `Cromossomo` (arquivo `cromossomo.py`), cujo atributo central é `_genes : Set[int]`: um **conjunto (set) de identificadores de colunas** selecionadas para compor a solução. As colunas que compõem a solução são identificadas diretamente pelo conteúdo do conjunto `_genes`: pertencer à solução equivale a pertencer a esse conjunto..

A cobertura de uma solução é verificada pelo método `validarSolucao()` da classe `Teste`, que constrói um vetor booleano de tamanho $m + 1$ (uma posição por linha, mais uma posição de guarda no índice 0), marca como `True` todas as linhas cobertas pela união das linhas cobertas por cada gene da solução, e verifica, por meio de um `reduce` lógico (AND), se todas as posições estão marcadas. Linhas não cobertas são armazenadas em `_linhasDescobertas`, permitindo diagnosticar exatamente quais linhas ficaram descobertas em caso de solução inválida.

3.4 Estruturas auxiliares

Além dos dicionários `pesoColunas`, `_linhasPorColuna` e `_colunasPorLinha` descritos anteriormente, que respondem, respectivamente, a “qual o custo da coluna j ?”, “quais linhas a coluna j cobre?” e “quais colunas cobrem a linha i ?”, o código implementa uma estrutura auxiliar específica para acelerar a eliminação de redundâncias e a busca local: o **vetor de redundância** (`_redundancias`, armazenado no próprio `Cromossomo`).

Esse vetor tem uma posição por linha e armazena, para cada linha i , o número de colunas da solução corrente que a cobrem. Ele é calculado pelo método `_calcularRedundancias()` e permite ao método `ehRedundante()` decidir, se uma coluna j pode ser removida da solução sem descobrir nenhuma linha, isto é, se todas as linhas cobertas por j possuem redundância ≥ 2 (cobertas por pelo menos mais uma outra coluna). Quando a coluna é removida

(`removeGene()`), o vetor de redundância é atualizado localmente, decrementando em 1 a redundância de cada linha coberta pela coluna removida, evitando o recálculo completo da estrutura.

Essa estrutura é utilizada tanto na eliminação de colunas redundantes (`_eliminarRedundancias()`, chamada após a construção de cada indivíduo, após o *crossover* e após a mutação) quanto, indiretamente, na busca local, cujo custo depende diretamente da eficiência das consultas de cobertura por coluna oferecidas por `_linhasPorColuna` e `pesoColunas`.

3.5 Operadores genéticos

Seleção

A seleção de reprodutores é feita pelo método `selecionarCromossomos()`, que implementa **seleção por ranking linear**. A população corrente é mantida permanentemente ordenada de forma decrescente de custo (do pior para o melhor indivíduo - ver “estrutura de população” abaixo), e o método `_rankearCromossomos()` atribui a cada indivíduo de posição k (com $k = 1$ para o pior e $k = N$ para o melhor, sendo N o tamanho da população) uma probabilidade de seleção:

$$p_k = \frac{2k}{N(N+1)}$$

que corresponde à conhecida fórmula de *ranking linear*, na qual a soma das probabilidades de toda a população é igual a 1 e indivíduos de melhor qualidade (maior k) recebem probabilidade proporcionalmente maior de serem selecionados. A seleção dos dois pais (`pai`, `mae`) é feita por amostragem ponderada com reposição (`random.choices(..., k=2)`), o que implica que, com probabilidade não nula, o mesmo indivíduo pode ser sorteado como pai e mãe simultaneamente.

Crossover

O operador de *crossover*, implementado em `cruzarCromossomos()`, é um **operador de união de genes**: o cromossomo filho recebe como genes a **união** dos conjuntos de colunas dos dois pais ($G_{\text{filho}} = G_{\text{pai}} \cup G_{\text{mae}}$). Como a união de duas soluções viáveis é sempre viável (nunca descobre uma linha que já estava coberta em qualquer um dos pais), esse operador garante viabilidade por construção, ao custo de gerar, tipicamente, soluções com mais colunas (e, portanto, mais redundância) do que os pais. Por esse motivo, o *crossover* está diretamente acoplado ao mecanismo de reparo descrito a seguir.

Mutação

A mutação (`mutarCromossomo()`) é **probabilística e aditiva**. A cada geração, sorteia-se $p \sim U(0, 1)$ e a mutação só é aplicada se p for menor que uma **taxa de mutação autoadapt-**

tativa taxaMutacao, calculada por:

$$\text{expoente} = -\frac{C_{\text{pior}} - C_{\text{melhor}}}{C_{\text{pior}}}, \quad \text{taxa} = \frac{\text{taxaMutMin}}{1 - e^{\text{expoente}}}$$

em que C_{pior} e C_{melhor} são, respectivamente, o custo do pior e do melhor indivíduo da população corrente. Quando a população está diversificada ($C_{\text{pior}} \gg C_{\text{melhor}}$), expoente é fortemente negativo, $e^{\text{expoente}} \rightarrow 0$ e a taxa tende a taxaMutMin. Quando a população converge ($C_{\text{pior}} \rightarrow C_{\text{melhor}}$), expoente $\rightarrow 0$, o denominador $1 - e^{\text{expoente}} \rightarrow 0$ e a taxa cresce sem limite (tratada como $+\infty$ no caso degenerado expoente = 0), forçando a mutação a ocorrer com probabilidade 1. Esse mecanismo é, portanto, um dispositivo de **escape de convergência prematura**: quanto mais homogênea a população, maior a pressão de mutação.

Quando efetivamente aplicada, a mutação (`_mutar()`) **adiciona** um número aleatório de colunas novas (sorteadas dentre as colunas ainda não presentes no indivíduo) ao conjunto de genes, entre 1 e $\lfloor u \cdot |G| \rfloor + 1$ novas colunas, com $u \sim U(0, 1)$ e $|G|$ o tamanho atual do genoma, sem remover nenhuma coluna existente. A mutação, portanto, nunca reduz diretamente o número de colunas; qualquer redução de tamanho decorre do reparo por eliminação de redundância aplicado em seguida.

Reparo

Não há um operador de reparo isolado e nomeado como tal, mas a função `_eliminarRedundancias` desempenha esse papel de forma integrada: ela é chamada automaticamente ao final da construção de cada indivíduo, do *crossover* e da mutação, percorrendo os genes do indivíduo em ordem aleatória e removendo, um a um, todo gene cuja remoção não descubra nenhuma linha (isto é, todo gene redundante, conforme o vetor de redundância). Esse procedimento garante que toda solução manipulada pelo AG permaneça viável e sem colunas manifestamente supérfluas, funcionando como um reparo/limpeza pós-operador.

Crítérios de parada

O critério de parada, implementado no laço principal em `source.py`, combina **dois critérios**:

1. um número máximo de gerações *bem-sucedidas* (`maxIter`), o contador `iter` só é incrementado quando a população é efetivamente atualizada por um filho melhor que o pior indivíduo corrente;
2. um critério de **estagnação**: um contador `counter` é incrementado a cada geração sem sucesso e reiniciado a cada sucesso; se `counter` atingir (`maxIter/4`) gerações consecutivas sem melhoria, o laço é interrompido antecipadamente, mesmo que `maxIter` não tenha sido atingido.

Ao final, a solução é validada (`validarSolucao()`) antes de ser reportada.

3.6 Procedimentos de busca local

A busca local é implementada pelo método `buscaLocal()` e é aplicada, no laço principal, **sobre o filho recém-gerado**, imediatamente após a mutação e antes da tentativa de inserção na população (etapa 5 do algoritmo em `source.py`).

Vizinhança: a vizinhança explorada é uma **vizinhança de troca 1-para-1 baseada em dominância de cobertura**: para cada coluna $j \notin S$ (fora da solução) e cada coluna $g \in S$ (na solução), verifica-se se $L_g \subseteq L_j$ (a coluna candidata cobre *pelo menos* todas as linhas que a coluna g cobre) e $c_j < c_g$ (a coluna candidata é mais barata). Se ambas as condições forem satisfeitas, a troca $g \rightarrow j$ é uma *melhoria estritamente dominante*: substitui-se uma coluna por outra que cobre um superconjunto das mesmas linhas a um custo menor, o que necessariamente reduz o custo total sem jamais descobrir qualquer linha.

Acionamento e estratégia: a busca implementada é do tipo **primeira melhoria** (*first-improvement*): assim que o primeiro par (g, j) que satisfaz as condições de troca é encontrado, os dois laços aninhados são interrompidos (`break`) e a troca é aplicada. Note-se, contudo, que a variável de controle `PARAR` é definida como `True` incondicionalmente ao final de cada execução do laço `while`, independentemente de uma melhoria ter sido encontrada ou não. Isso significa que `buscaLocal()` realiza **no máximo uma única troca por chamada**, não há iteração até um ótimo local da vizinhança; a busca local não é re-executada internamente após aplicar uma melhoria. A intensificação ao longo de várias gerações ocorre, portanto, de forma incremental: como o método é chamado uma vez por geração sobre cada novo filho, sucessivas gerações podem aplicar sucessivas trocas de melhoria, mas cada chamada individual está limitada a um único movimento.

Interação com o AG: após uma troca bem-sucedida, o vetor de redundância do indivíduo é recalculado e `_eliminarRedundancias()` é chamada novamente, de modo que uma melhoria de busca local pode, por sua vez, tornar outras colunas redundantes e disparar sua remoção. O indivíduo resultante da busca local é o que efetivamente disputa uma vaga na população via `atualizaPopulacao()`.

3.7 Estrutura de dados da população e substituição

A população (`_populacaoAtual`) é mantida, a cada momento, como uma lista **ordenada de forma decrescente por custo** (do pior indivíduo, no índice 0, ao melhor, no último índice), por meio de inserção ordenada com busca binária (`bisect.bisect`) no método `_insereIndividuo()`. Essa escolha evita reordenar toda a população a cada geração e permite obter o melhor e o pior indivíduo em tempo $O(1)$ (métodos `getCustoMaisApto()` e `getCustoMenosApto()`).

A substituição de indivíduos (`atualizaPopulacao()`) segue um esquema **estacionário e elitista** (*steady-state*): a cada geração, apenas um filho é gerado; ele só é inserido na população, em substituição ao pior indivíduo corrente (`insereFilho()`), se seu custo for estritamente menor que o custo do pior indivíduo da população. Esse esquema garante que o melhor indivíduo encontrado nunca seja perdido e que a qualidade média da população seja monotonicamente não decrescente ao longo das gerações bem-sucedidas.

3.8 Elementos de originalidade

Em atendimento à exigência do enunciado de que a implementação apresente elementos de originalidade e não seja mera reprodução de um algoritmo publicado, destacam-se os seguintes componentes não triviais da implementação:

- **Algoritmo de construção aleatório:** O algoritmo escolhe as colunas que irão compor a solução de um indivíduo original de maneira aleatória.
- **Vizinhança de busca local baseada em dominância de cobertura:** a vizinhança explorada pela busca local é uma vizinhança especificamente desenhada para o SCP, que só aceita trocas em que a coluna candidata cobre um superconjunto das linhas cobertas pela coluna removida a um custo estritamente menor.
- **Critério de parada híbrido (meta de gerações + estagnação relativa):** o algoritmo combina um número máximo de gerações bem-sucedidas com um critério de estagnação proporcional a esse máximo (`maxIter/4`), permitindo término antecipado sem a necessidade de um parâmetro adicional independente para a paciência do algoritmo.

4 Experimentos Computacionais

4.1 Ambiente de execução

- **Linguagem:** Python 3, versão 3.10 ou superior.
- **Bibliotecas utilizadas:** apenas bibliotecas padrão da linguagem para a lógica do algoritmo (`random`, `bisect`, `math`, `json`, `re`, `csv`, `functools`, `time`); `matplotlib` e `pandas` para geração de gráficos e tabelas de resultados (módulo `graficos.py`).

4.2 Instâncias utilizadas

Foram utilizadas as nove instâncias fornecidas pelo professor, cujos valores de referência (obtidos de `referencia.json`) são apresentados na Tabela 1.

Tabela 1. *Instâncias utilizadas e respectivos valores de referência.*

Instância	Valor de referência
test01.dat	557,44
test02.dat	537,89
test03.dat	517,58
test04.dat	1 162,80
test05.dat	1 020,12
wren01.dat	7 856,00
wren02.dat	13 908,00
wren03.dat	13 780,00
wren04.dat	58 161,00

4.3 Parâmetros do AG e da busca local

Os parâmetros de configuração do algoritmo e seus valores adotados para a realização dos testes são:

- Tamanho da população: 100 indivíduos;
- Taxa de mutação mínima (taxaMutMin): 1,00;
- Número máximo de gerações (numIter): 250;
- Número de execuções independentes por instância: 10.

A busca local não possui parâmetros próprios configuráveis: por ser executada uma única vez por geração, sobre o filho recém-criado, sua “intensidade” está diretamente acoplada ao número de gerações do AG.

4.4 Processo de calibração dos parâmetros

Para calibrar os parâmetros, foi feita uma bateria de testes com o arquivo "test01.dat" a fim de se investigar a influência do tamanho da população, do número de iterações e da taxa de mutação mínima sobre a qualidade do resultado e sobre o tempo de execução. Ao todo, foram testadas 9 configurações distintas, cada uma sendo executada 5 vezes sobre o arquivo "test01.dat".

populações	100, 200, 300
taxas de mutação	0.5, 0.75, 1.00
iteraões	100, 175, 250

Os resultados obtidos foram os seguintes:

A Figura 1 permite concluir que os tamanhos escolhidos para a população inicial não influenciaram positivamente na construção de melhores resultados. Com base nisso, foi-se escolhido como tamanho padrão 100 indivíduos.

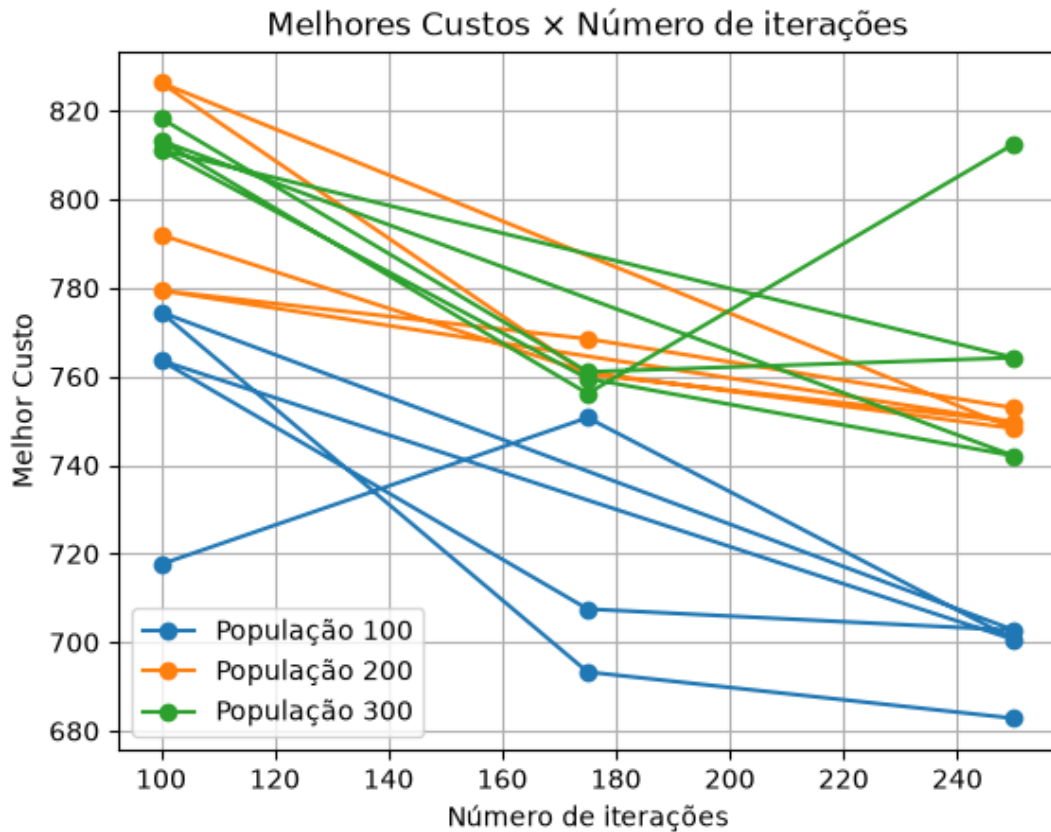


Figura 1. *Relação de custo por iteração*

A Figura 2 permite concluir a taxa mínima de mutação com melhor influencia sobre a qualidade das soluções geradas foi 1.0 e essa influencia foi maior quando a população continha 100 indivíduos.

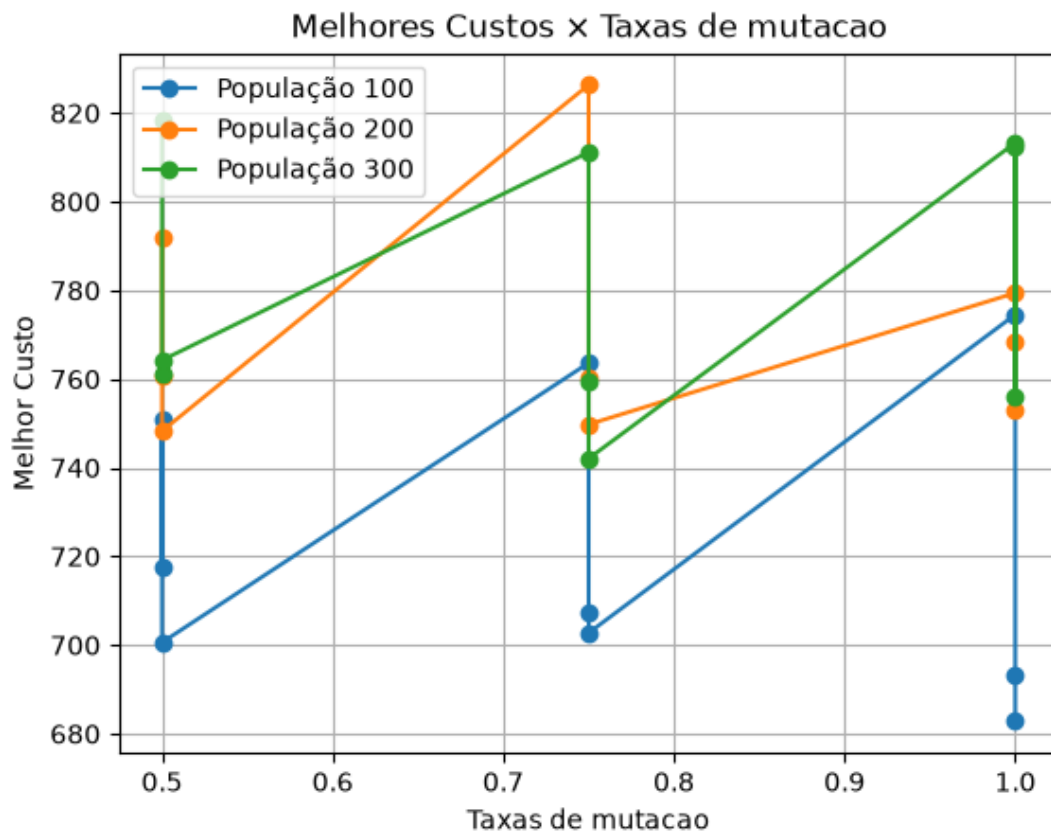


Figura 2. *Relação de custo por taxa de mutação*

A Figura 3 permite verificar que o número de iterações máximas (número de gerações) impacta diretamente no tempo de execução do algoritmo, impactando menos quanto menor foi o tamanho da população.

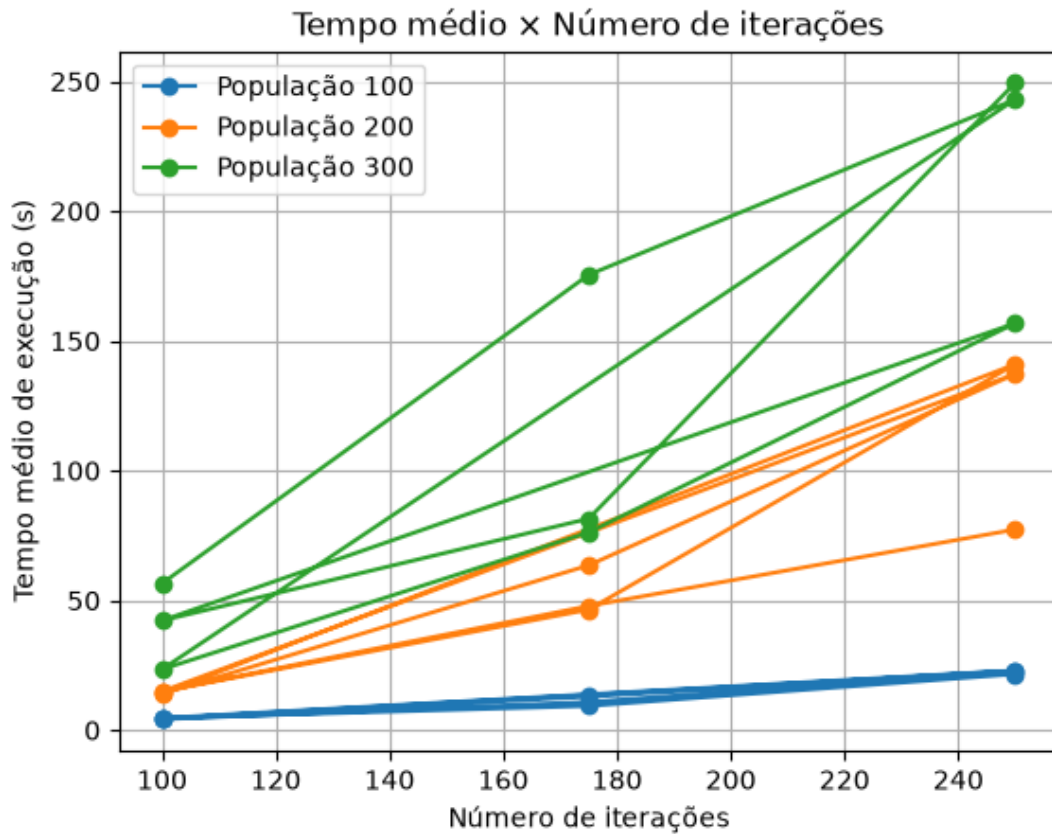


Figura 3. *Relação de tempo por iterações*

Assim, de acordo com os resultados da análise, ficou evidente que a melhor escolha de configuração era:

- **População:** 100 indivíduos,
- **Taxa Mínima de Mutação:** 1.0
- **Numero de Iterações:** 250

5 Resultados

5.1 Melhores soluções, custos, tempos e GAP

A Tabela 2 apresenta, para cada instância, os resultados das 10 execuções independentes, o tempo médio de processamento, o GAP em relação ao valor de referência e o GAP médio das 10 execuções. O GAP é calculado, seguindo exatamente a fórmula implementada em `graficos.py`, como:

$$\text{GAP} = \frac{\text{custo obtido} - \text{valor de referência}}{\text{valor de referência}}$$

(GAP com sinal, e não em módulo, uma solução melhor que a referência resultaria em GAP negativo, embora isso não tenha ocorrido em nenhuma das execuções realizadas).

Tabela 2. Resultados dos testes computacionais

Problema	A melhor solução de cada teste										Melhor Solução ^a	GAP	Tempo de execução ^b
	1	2	3	4	5	6	7	8	9	10			
Teste 1	754.86	715.22	787.79	-	742.36	763.79	760.82	712.28	708.08	755.79	696.06	24.86	19.03
Teste 2	820.50	779.56	770.50	763.73	-	763.77	716.06	761.50	763.38	807.90	701.29	30.37	17.97
Teste 3	749.89	748.14	751.07	710.06	765.06	742.08	-	758.48	735.79	735.35	681.69	31.70	16.40
Teste 4	1613.80	1514.18	1498.32	1525.17	1526.73	-	1529.75	1492.87	1510.98	1517.73	1489.97	28.13	18.38
Teste 5	1313.90	1372.58	1340.77	1339.28	1331.27	1330.22	1306.18	1300.50	1343.15	-	1284.53	25.91	19.56
Wren 01	9457.00	9147.00	9489.00	8967.00	9133.00	9179.00	8978.00	8972.00	9485.00	-	8941.00	13.81	42.17
Wren 02	18437.00	-	18133.00	18785.00	18469.00	18281.00	18005.00	18317.00	18639.00	18456.00	17951.00	29.06	135.63
Wren 03	18903.00	18490.00	18266.00	18900.00	18779.00	-	19043.00	19093.00	18396.00	18391.00	18150.00	31.71	55.07
Wren 04	-	74863.00	74555.00	74728.00	74518.00	73699.00	73808.00	73948.00	74953.00	73897.00	73133.00	25.74	119.18
Soma											26.81	443.39	

^a Melhor solução alcançada nos testes.

^b Tempo médio de execução em segundos.

Observa-se que o algoritmo obteve GAPs entre aproximadamente 14% (wren01) e 32% (test03/wren03) em relação aos valores de referência, e que o tempo de processamento cresce de forma acentuada com o porte da instância, especialmente nas instâncias wren02, wren03 e wren04, que possuem maior número de linhas e colunas que as instâncias testXX.

5.2 Boxplot do GAP por instância

A Figura 4 apresenta a distribuição do GAP (em %) das 10 execuções independentes para cada uma das nove instâncias avaliadas.

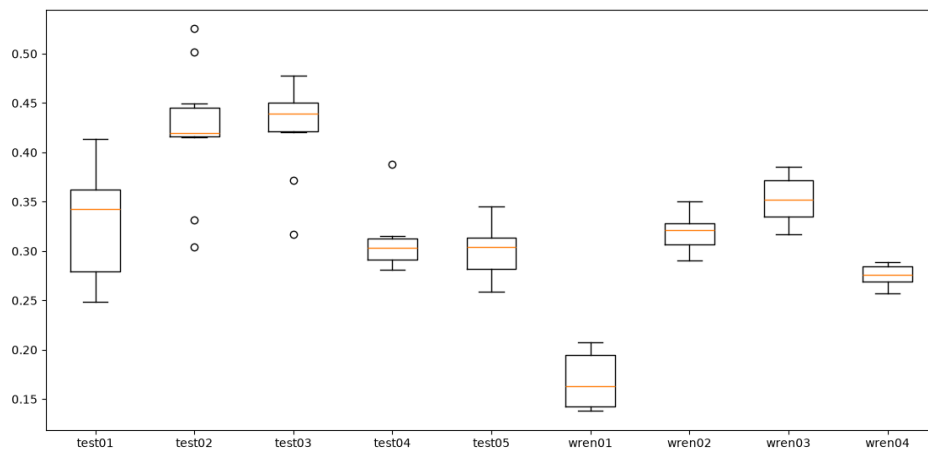


Figura 4. Boxplot do GAP (%) para cada instância, com base em 10 execuções independentes (população = 100, taxa de mutação mínima = 1,00, iterações = 250).

5.3 Análise de convergência

As Figuras 5, 6 e 7 apresentam a evolução do custo do melhor e do pior indivíduo da população ao longo das gerações, para a instância `test01.dat` com 100 indivíduos e `wren04.dat` com 100 e 200 indivíduos respectivamente. Para facilitar a visualização, os dados do melhor e pior indivíduo foram coletados a cada 10 atualizações da população (que só se atualiza quando um indivíduo melhor do que o pior é gerado).

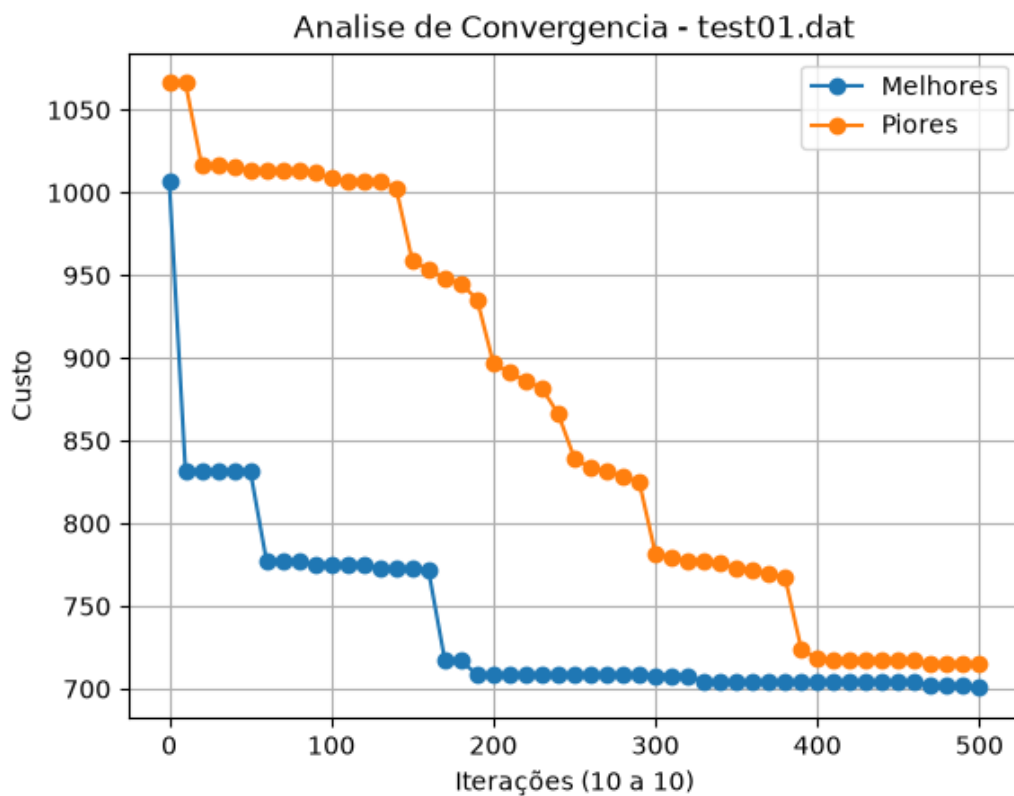


Figura 5. Evolução do custo do melhor e do pior indivíduo da população ao longo das gerações, instância `wren04.dat`.

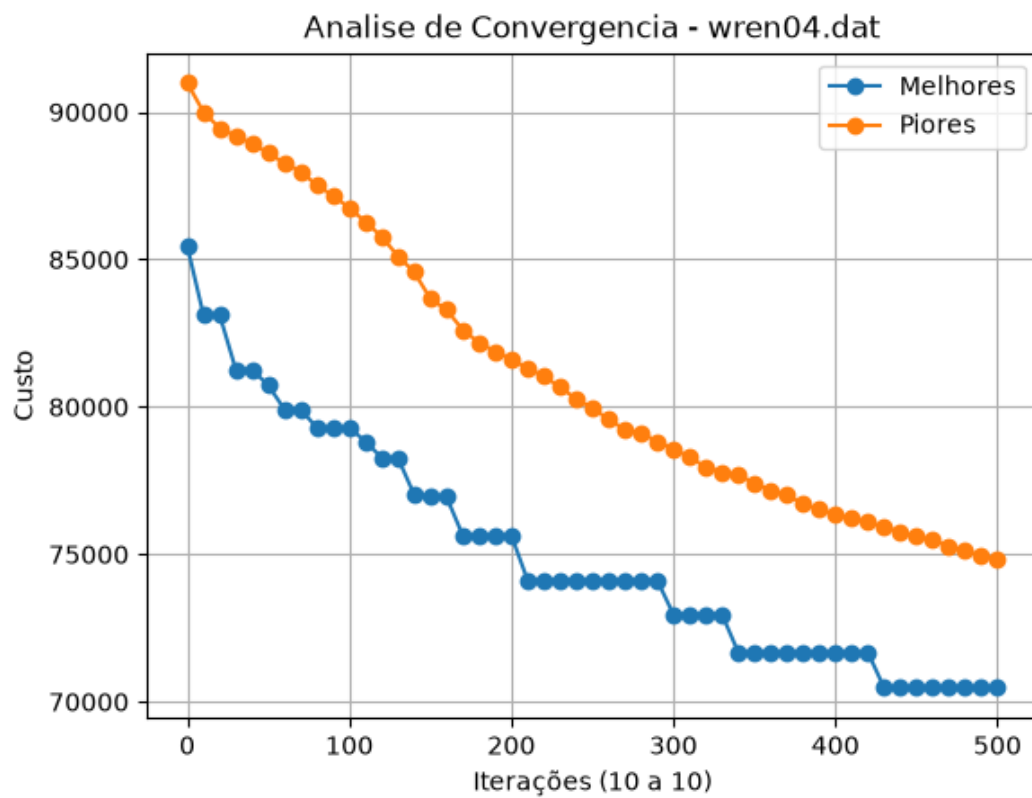


Figura 6. Evolução do custo do melhor e do pior indivíduo da população ao longo das gerações, instância wren04.dat.

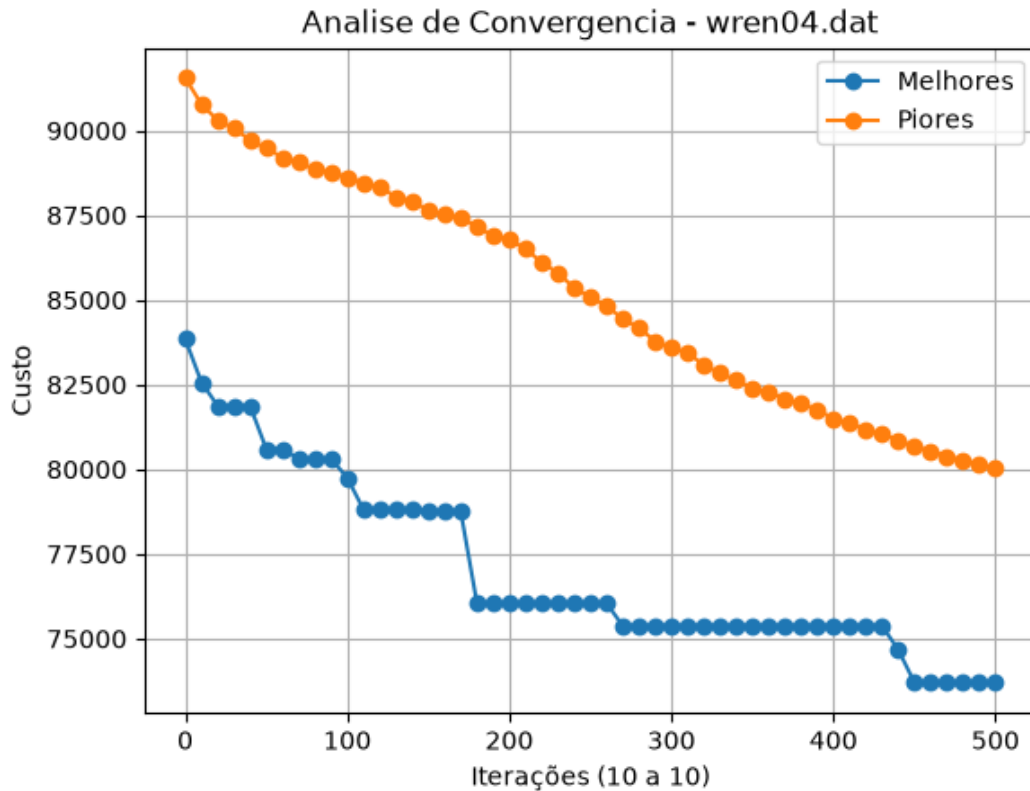


Figura 7. Evolução do custo do melhor e do pior indivíduo da população ao longo das gerações, instância *wren04.dat*.

O gráfico evidencia o comportamento esperado de um esquema estacionário elitista: o custo do melhor indivíduo é monotonicamente não crescente (nunca piora, uma vez que só é substituído por soluções melhores), com quedas em degraus concentradas em certas faixas de gerações, intercaladas por longos períodos de estagnação aparente do melhor indivíduo, períodos em que a busca local e a mutação seguem atuando sobre a população sem, contudo, produzir um filho melhor que o pior indivíduo corrente. O custo do pior indivíduo decresce de forma mais suave e contínua, refletindo a renovação gradual da população pelo esquema de substituição do pior elemento. A distância entre as curvas de melhor e pior custo diminui ao longo da execução, indicando convergência (redução de diversidade) da população, o que, por sua vez, é o gatilho para o aumento da taxa de mutação autoadaptativa descrita na Seção 3.

6 Conclusões

O algoritmo genético híbrido implementado, combinando construção randomizada, operadores de seleção por ranking linear, *crossover* por união com reparo, mutação aditiva autoadaptativa e busca local por dominância de cobertura, mostrou-se capaz de produzir soluções viáveis para todas as nove instâncias avaliadas, com GAPs em relação aos valores de referência situados aproximadamente entre 14% e 32%. Esses valores indicam que o algoritmo converge

para soluções de qualidade razoável, porém ainda distantes dos melhores valores conhecidos na literatura, sobretudo se comparados a implementações de referência para o SCP que costumam atingir GAPs de poucos pontos percentuais.

O impacto das decisões de implementação discutidas na Seção 3 é relevante para explicar esse resultado. Em particular, o método randômico de construção dos primeiros indivíduos, reduz a qualidade da população inicial, exigindo maior esforço do AG e da busca local para compensar essa perda de qualidade nas gerações subsequentes. De modo semelhante, o fato de a busca local realizar no máximo uma única troca por chamada (em vez de iterar até um ótimo local da vizinhança) limita a capacidade de refinamento de cada indivíduo por geração, distribuindo esse refinamento ao longo de várias gerações em vez de concentrá-lo.

A taxa de mutação autoadaptativa parece cumprir seu papel de reagir à convergência da população, como evidenciado indiretamente pelo comportamento em degraus da curva de convergência (Figura 5).

Quanto ao desempenho computacional, o tempo de execução cresce de forma acentuada com o porte da instância (de cerca de 15–19 segundos para as instâncias *testXX*, de menor porte, a mais de 150 segundos para *wren02*), o que é esperado dado que diversas operações do algoritmo (em particular a busca local, que itera sobre pares coluna-fora $\times$ gene-dentro-da-solução) têm custo dependente do número de colunas e do tamanho da solução corrente.

Limitações identificadas

- A busca local está limitada a uma única troca por chamada, não iterando até um ótimo local da vizinhança;
- Não há comparação experimental entre diferentes estratégias de busca local ou estruturas de vizinhança, item explicitamente solicitado nos critérios de avaliação;

Possíveis melhorias

- Estender `buscaLocal()` para iterar até a ausência de melhorias na vizinhança (busca local completa), ou alternar explicitamente entre estratégias *first-improvement* e *best-improvement* como objeto de comparação experimental;
- Implementar e comparar vizinhanças adicionais de busca local (por exemplo, trocas *k*-para-1, ou uma combinação de remoção gulosa seguida de reinserção);

Referências

CONSTANTINO, A. A. et al. Aplicação de algoritmos genéticos ao problema de cobertura de conjunto. In: **Anais do XXXV Simpósio Brasileiro de Pesquisa Operacional (SBPO)**. Natal, RN: SOBRAPO, 2003. Departamento de Informática, Universidade Estadual de Maringá.